

Distributed Conductance Testing with Random Walks

Summary for a directed study under Professor Dora Erdős in Spring 2026 @ Boston University

1. Introduction

This paper summarizes what I learned during a directed study on distributed property testing for graph conductance, in collaboration with and supervised by Professor Dora Erdős at Boston University. The study centered on two CONGEST model algorithms that test whether a graph is a good “conductor” (well-mixed, expander-like) or is not, using only short random walks and $O(\log n)$ rounds of communication: Fichtenberger & Vasudev's two-sided error tester (MFCS 2018) and Batu, Trehan & Trehan's simplified single-phase tester. Alongside reading and working through many of the theorems presented in the two papers with Professor Erdős, I built a shared Python implementation of both algorithms, a synthetic “graph zoo” of good and bad conductors, an experiment harness, and a visualization pipeline that renders how each algorithm's random walks actually spread across a graph. These implementations were done in collaboration with Claude code and Cursor AI LLM tools. Though much of the implementation was done by myself, large portions of the test suite and many of the comments were created by Cursor and I actively used Claude code to aid in the debugging process and to aid in the implementation of the aggregate functions. This document covers the concepts, compares the two algorithms, and reports what the implementation revealed, including an implementation specific detail between the two testers' practical behavior that was not obvious to me from reading the papers alone.

2. Background: CONGEST

The conductance of a graph measures how well-connected it is: informally, $\phi(G)$ is the smallest ratio of edges crossing a cut to the volume of the smaller side of that cut, minimized over all cuts. A high-conductance graph mixes quickly under a random walk (like an expander); a low-conductance graph has a bottleneck i.e. a large region that is only weakly connected to the rest of the graph (like two cliques joined by a single bridge edge).

A property tester for conductance is a randomized algorithm that, given parameters Φ and ϵ , accepts graphs with conductance at least Φ and rejects graphs that are ϵ -far (need $> \epsilon m$ edge edits) from having some related, smaller conductance, with probability $\geq 2/3$. Both papers work in the CONGEST model of distributed computing: every vertex is a processor, computation proceeds in synchronous rounds, and each edge can carry only $O(\log n)$ bits per round. Both testers follow from the same idea, formalized independently for the sequential setting by Kale & Seshadhri: random walks mix quickly on good conductors and get “trapped” in low-conductance regions on bad ones, so watching where short random walks end up is enough to distinguish the two cases. The presented challenge is doing this without violating the CONGEST bandwidth limit.

3. Fichtenberger & Vasudev (2018)

Fichtenberger and Vasudev (from here on referred to as F&V)'s tester is a two-sided error algorithm with round complexity $O(\log n / \Phi^2)$, it is the first CONGEST algorithm for this problem, and it has a matching $\Omega(\log n)$ lower bound. It runs in five phases:

- **Phase 0:** Diameter check: build a BFS spanning tree up to depth $O(\log n / \Phi)$; reject immediately if it does not reach every vertex, since good conductors must have small diameter.
- **Phase 1:** Count edges: convergecast vertex degrees up the BFS tree to compute m , then broadcast it back down.
- **Phase 2:** Sample sources: each vertex marks itself as a source with probability proportional to its degree (volume-weighted sampling), producing a small source set S .
- **Phase 3:** Random walks: from each source, run $N = n^{100}$ lazy random walks of length $\ell = O(\log n / \Phi^2)$. To avoid congestion, vertices never forward individual walk identities, only aggregated counts “(source, count) walks are here” per edge per round.
- **Phase 4:** Aggregate discrepancy: for each source v , convergecast the squared deviation of the endpoint distribution from the stationary distribution π , producing a single number s_v per source.
- **Phase 5:** Decision phase, accept iff every s_v is at most a tiny threshold (m^{-15} in the paper); otherwise reject. Because every vertex sees the same aggregated values, all vertices reach the same decision.

The correctness proof uses a spectral decomposition of the normalized Laplacian to show that if a large-volume, low-conductance cut exists, a constant fraction of its volume consists of “weak” vertices whose walk distribution stays measurably far from stationary and degree-weighted sampling hits such a vertex with good probability.

4. Batu, Trehan & Trehan — “All You Need are Random Walks”

The BTT paper goes a step farther and asks: does Phase 4's global aggregation (the spanning tree, convergecast, and broadcast) actually serve as a necessary component? Their answer is no. Their tester achieves the same optimal $O(\log n / \alpha^2)$ round complexity using only two phases and no spanning tree at all:

- **Phase 1:** Sample sources: identical degree-weighted sampling as F&V, but done without any global structure.
- **Phase 2:** Random walks + local decision: run $K = 2m^2$ lazy walks of length ℓ from each source, again propagating only aggregated (source, count) tuples to avoid congestion. Each vertex v locally compares, for every source q , the number of walks from q that ended at v against a threshold $\tau_v = m \cdot \deg(v) \cdot (1 + 2n^{-1/4})$. If any count exceeds τ_v , v rejects with no communication with any other vertex is needed to make this decision.

The idea is “sticky vertices”: BTT shows that any large low-conductance region contains a substantial subset of vertices from which a short walk is likely to return to (a subset of) that same region and not just remain close to it in the sense F&V uses, but literally revisit it, which is what a purely local vertex-count check can detect. Removing the spanning tree also removes its single point of failure as F&V's decision depends on every internal tree node surviving Phase 4, while BTT's decision at each vertex depends only on walks that have already terminated there. In short, with the number of vertices and nodes we have- is there one vertex that is “catching” more walks than we would expect (in a well connected graph)?

5. Comparing the Two Algorithms (chart generated by Claude)

Aspect	Fichtenberger & Vasudev (2018)	Batu, Trehan & Trehan
Round complexity	$O(\log n / (\epsilon \Phi^2))$	$O(\log n / \alpha^2)$
Walks per source	$N = n^{100}$	$K = 2m^2$
Spanning tree	Required (BFS, for aggregation)	Not used
Aggregation	Convergecast to root, then broadcast	None, purely local
Phases	5 (diameter, count, sample, walk, aggregate/decide)	2 (sample, walk/decide)
Decision statistic	Global: squared deviation from stationary π , per source	Local: raw walk count vs. a degree-scaled threshold, per vertex
Fault tolerance	Fragile — one failed internal tree node breaks aggregation	Robust — no shared infrastructure to fail
Conductance gap proved	Φ vs. $\Phi^2/768$ (or $\Phi^2/1000$ in the journal statement)	α vs. $\alpha^2/2880$

Both papers prove the same asymptotic round complexity and both reduce to “run random walks, send only counts.” The contribution of the second paper is architectural as it shows the spanning-tree aggregation phase is not necessary for correctness, at the cost of a slightly worse (but still constant) gap between the accept and reject thresholds. This is the kind of result that is easy to state but non-obvious to prove, since it requires a new spectral argument (the “trap probability” / sticky-vertex analysis) rather than reusing F&V’s discrepancy-from-stationary argument.

6. Implementation and Visualization

Alongside the reading, we built a shared Python codebase implementing both testers from their pseudocode, plus infrastructure to compare them on the same graphs (with assistance from LLM tools as mentioned above):

- `btt.py` / `fv.py` — direct, per-walk simulations of Algorithm 1 (and its sub-procedures) from each paper, closely mirroring the papers’ notation (ℓ , K , N , τ_v , S , Q , etc.), each with a visualization mode.

- `btt_aggregated.py` / `fv_aggregated.py` — count-propagation rewrites of both testers. Instead of simulating each of the K or N walks individually, these propagate a count vector through a multinomial split at every step (falling back to a Gaussian approximation for counts too large for exact multinomial sampling). This reproduces the same random process the CONGEST algorithm actually runs — it is the aggregation trick from Algorithm 2 in both papers — but lets us reach the papers' true walk budgets (up to $N = n^{100}$) in practice.
- `graph_zoo.py` — constructors for a set of good conductors (complete graphs, random regular graphs, dense Erdős–Rényi graphs) and bad conductors (barbell graphs, lollipops, two cliques joined by k bridge edges, stochastic block models, paths), used consistently across all experiments.
- `experiments.py` — four experiments: E1 correctness across the whole graph zoo, E2 scaling with n , E3 sensitivity to the walk budget, and E4 a conductance-gap ladder (sweeping the number of bridge edges between two cliques from 1 up to the clique size).
- `plot_from_csv.py` — renders a figure per (algorithm, graph) from saved node/edge CSVs: each vertex is colored by how many walks ended there, with a red ring marking vertices that locally reject (BTT) and a black ring marking source vertices (F&V).

7. What the Implementation Showed

The two algorithms behave very differently once you actually run them, in a way the papers' asymptotic guarantees do not make obvious.

7.1 F&V works, but only once its walk count is made computationally feasible

For a bit of an anecdote, Professor Erdos told me about a meeting she had with a colleague where she mentioned F&V's suggested n^{100} walks- a number he literally laughed at. But as we found through the implementation, at a tractable per-walk sample size (N around n^2), F&V's discrepancy statistic is dominated by sampling noise: every graph, good or bad, produces a discrepancy of roughly $1/N$, because each estimate $\hat{W}$ is itself a $\text{Binomial}(N, \cdot)/N$ quantity with variance of that order. The paper's $N = n^{100}$ exists to push that noise floor far below the true signal, but simulating n^{100} individual walks is obviously infeasible (hence the laughter). Once I switched to the count-aggregated implementation (which reproduces the same distribution exactly, without a per-walk loop) F&V reached the paper's intended regime and worked almost perfectly: 100% decision accuracy across every graph in our zoo (11 graphs $\times$ 20 trials), across every scaling test up to $n = 320$, and across the full conductance-gap ladder in the regime where the expectation matched the algorithm's actual accept/reject boundary.

7.2 BTT, as specified in the paper, is too conservative to run at practical graph sizes

BTT's per-vertex threshold check should have detected our “bad” graphs (barbell graphs, two cliques joined by a thin bridge, stochastic block models) just as reliably. Instead, at every graph size I first tried ($n \approx 20$ –80) and at every walk budget I tried, including the full, paper-faithful $K = 2m^2$ (once the aggregated implementation made that budget computationally reachable) BTT accepted almost every bad graph in the zoo. This apparent bug caused the majority of my anguish in this implementation. But when I realized it may not actually be a bug, investigating why this was the case led to the most interesting result of the study: the completeness slack term in the paper's threshold, $\tau_v = m \cdot \deg(v) \cdot (1 +$

$2n^{-1/4}$), is an asymptotic statement. The $2n^{-1/4}$ term is designed to vanish as $n \rightarrow \infty$, but at $n = 40$ it equals about 0.67 (a 67% margin on top of the expected walk count) and that margin alone was larger than the gap between a good and a bad graph's walk counts at that size. I confirmed this by observing that the ratio of the largest observed vertex count to its threshold τ_v sat around 0.55–0.60 for good graphs and only 0.58–0.89 for most bad graphs in the zoo, an almost 1:1 overlap, and stayed below 1 (i.e., BTT accepted) even at the paper's full walk budget. I then tested the same bad-graph family at increasing size and watched the ratio climb monotonically through the threshold: 0.53 at $n = 20$, 0.58 at $n = 40$, 0.71 at $n = 60$, 0.83 at $n = 80$, then crossing 1.0 to correctly reject at $n = 120$ and $n = 160$. This is not a bug in the code but rather it is a demonstration that BTT's proven conductance gap (α vs. $\alpha^2/2880$) only becomes visible in a local threshold statistic once n is large enough for the finite-size slack term to shrink out of the way, something that is invisible from the asymptotic statement of the theorem, and that I would not have thought to investigate without actually implementing and sweeping over the algorithm.

7.3 Aggregation is what makes either algorithm testable in practice

A second, related lesson was purely about tooling: the naive per-walk simulations in `btt.py` and `fv.py` are correct but their cost scales with the raw walk count (K or N), which the papers deliberately set at enormous numbers (remember the laughter....). The count propagation rewrites (`btt_aggregated.py`, `fv_aggregated.py`) were the difference between theoretically testing the paper's claims and actually generating results. For F&V this was required just to see a signal at all; for BTT it was required to reach the walk budget large enough to demonstrate the $n \rightarrow \infty$ crossover above without the per walk simulation taking hours per graph.

7.4 Visualization

Rendering the per-vertex walk count heatmaps for BTT on a barbell graph at small n visibly shows walks spilling evenly across the bridge into the “wrong” clique, rather than piling up on the source side the way the sticky-vertex intuition predicts, which is what the finite-size effect the numbers above quantify. On F&V, the same visualization shows the walk mass concentrating tightly near the stationary (degree-proportional) distribution on good conductors and visibly skewing toward the source's own region on bad ones.

8. Conclusion

Reading the two papers side by side made the architectural argument of the second (BTT) paper seem inevitable, as once you see F&V's spanning-tree aggregation, the natural question is whether it is really necessary, and BTT's sticky-vertex argument answers that. But implementing both algorithms end-to-end revealed an asymptotically-correct local decision rule can be practically powerless at the sizes anyone would actually simulate, while a global aggregated statistic can be robust almost immediately once its walk budget is made computationally feasible. The distinction between an algorithm being correct in the limit and an algorithm being useful at a size you can run was the most valuable thing I took from this directed study, and it is something I would not have found from the papers or the pseudocode alone. Implementing the actual pseudocode was also a fun challenge and encouraged me to delve further into the mathematical basis of the papers. I also gained a lot of experience using LLM based coding tools for debugging, and implementing certain functions. Using the

LLM to get ideas for how to create the aggregate functions was also an interesting experience (as was fixing some of its hallucinations). Overall I learned a lot about conductance, Python implementation specifics, and agentic coding tools! This was my first time doing something like this and I hope to continue to learn more about graph networks and theoretical computer science.